Privacy-Enhanced Synthetic Data Generation for Autonomous Vehicle Training

R26-CS-010

Design Document

B.Sc. (Hons) Degree in Information Technology Specialized in Cyber Security

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

Supervisor	**Prof. Anuradha Jayakody**
Co-Supervisor	**Mr. Dinith Primal**

S. A. S. D. Priyadarshi	IT22309556	Privacy Integration
Yubitha Sivagnanam	IT22541284	Dataset Generation and Simulation
M. K. S. R. Apeksha	IT22110220	Utility Evaluation
R. D. P. T. Wijesinghe	IT22066916	Risk and Compliance

May 2026

TABLE OF CONTENT

TABLE OF CONTENT 2
LIST OF TABLES 4
LIST OF FIGURES 5
LIST OF APPENDICES 6
1. INTRODUCTION 7
1.1 Purpose 7
1.2 Scope 7
1.3 Definitions, Acronyms, and Abbreviations 8
1.4 Overview 9
2. OVERALL DESCRIPTION 10
2.1 Product Perspective 10
2.1.1 System Interfaces 10
2.1.2 User Interfaces 11
2.1.3 Hardware Interfaces 11
2.1.4 Software Interfaces 11
2.1.5 Communication Interfaces 11
2.1.6 Memory Constraints 12
2.1.7 Operations 12
2.1.8 Site Adaptation Requirements 12
2.2 Product Functions 12
2.3 User Characteristics 13
2.4 Constraints 14
2.5 Assumptions and Dependencies 14
2.6 Apportioning of Requirements 15
3. SPECIFIC REQUIREMENTS 16
3.1 External Interface Requirements 16
3.1.1 User Interfaces 16
3.1.2 Hardware Interfaces 16
3.1.3 Software Interfaces 17
3.1.4 Communication Interfaces 17

3.2 Architectural Design 17
3.2.1 High Level Architectural Design 17
3.2.2 Hardware and Software Requirements with Justification 19
3.2.3 Risk Mitigation Plan with Alternative Solution Identification 20
3.2.4 Cost Benefit Analysis for the Proposed Solution 22
3.3 Performance Requirements 23
3.4 Design Constraints 24
3.5 Software System Attributes 25
3.5.1 Reliability 25
3.5.2 Availability 26
3.5.3 Security 26
3.5.4 Maintainability 26
3.6 Other Requirements 27
3.6.1 Privacy Requirements 27
3.6.2 Regulatory Requirements 27
3.6.3 Reproducibility Requirements 28
4. REFERENCES 29
5. APPENDICES 30
Appendix A - Module Interface Specification 30
Appendix B - Privacy Budget Allocation 31
Appendix C - SDG Alignment 31
Appendix D - Repository Structure 31

LIST OF TABLES

Table 1-1: List of Abbreviations 8
Table 2-1: Product Comparison 10
Table 3-1: Primary Entry Points 16
Table 3-2: The Pipeline Stages 17
Table 3-3: Hardware and Software Requirements 19
Table 3-4: Risk Mitigation Plan 20
Table 3-5: Project Research Costs 22

LIST OF FIGURES

Figure 3-1: High Level Architecture..... 19

LIST OF APPENDICES

Appendix A: Module Interface Specification 31
Appendix B: Privacy Budget Allocation 31

1. INTRODUCTION

1.1 Purpose

This Design Document describes the system architecture, component design, hardware and software configuration, risk mitigation strategy, and cost-benefit analysis for the Privacy-Enhanced Synthetic Autonomous Vehicle Data Generation Pipeline developed under project R26-CS-010.

The intended readers of this document are the academic supervisors who will evaluate the design decisions, the four group members who will implement each system component, data protection regulators who may need to verify that architecture enforces privacy by design, and future researchers who may replicate or extend the system.

This document covers the complete system design across all four components: the Privacy Integration module owned by IT22309556, the Dataset Generation and Simulation module owned by IT22541284, the Utility Evaluation module owned by IT22110220, and the Risk and Compliance module owned by IT22066916.

1.2 Scope

The system described in this document is a four-module software pipeline. It takes aggregate statistical properties derived from real-world autonomous vehicle driving data, applies epsilon-differential privacy mechanisms at the calibration and training stages, generates synthetic multi-sensor AV training data through the CARLA simulation environment, evaluates the utility of that data for AV perception model training, and validates the privacy guarantees through empirical membership inference attack testing.

The system is designed to address the absence of formal privacy enforcement in existing AV data generation platforms. Current platforms such as CARLA, AirSim, and NVIDIA Omniverse generate realistic synthetic sensor data but do not protect the calibration process that configures them from real driving data. This leaves a known attack surface that allows adversaries to extract information about individuals from the calibration inputs. This system applies the Laplace mechanism to the calibration step and Opacus DP-SGD to the generative model training step, providing a documented, measurable privacy guarantee at both points in the pipeline.

The two target end-user groups are AV development companies and research organizations that require GDPR-compliant synthetic training data, and academic researchers working on privacy-preserving machine learning for autonomous systems.

1.3 Definitions, Acronyms, and Abbreviations

Table 1-1: List of Abbreviations

Abbreviation	Full Form
epsilon-DP	Epsilon-Differential Privacy. A mathematical framework that limits what an observer can learn about any individual from the output of a computation. Parameterized by epsilon.
RDP	Renyi Differential Privacy. A tighter privacy accounting method used in Opacus that provides smaller epsilon bounds than the older moments accountant.
DP-SGD	Differentially Private Stochastic Gradient Descent. A training algorithm that clips per-sample gradients and injects Gaussian noise before weight updates.
MIA	Membership Inference Attack. An adversarial test that attempts to determine whether a specific record was included in a model's training set.
GAN	Generative Adversarial Network. A generative model architecture consisting of a generator and discriminator trained in competition.
VAE	Variational Autoencoder. An alternative generative model architecture used in data generation tasks.
mAP	Mean Average Precision. The primary utility metric for evaluating AV object detection model performance.
FID	Fréchet Inception Distance. A metric that measures statistical similarity between real and synthetic image distributions.
CARLA	Car Learning to Act. An open-source AV simulation platform developed by Dosovitskiy et al. (2017).
GDPR	General Data Protection Regulation. EU data protection legislation (Regulation (EU) 2016/679).
PDPA	Personal Data Protection Act No. 9 of 2022. Sri Lanka data protection legislation.
Opacus	A PyTorch library that implements epsilon-DP training with an RDP accountant, developed by Yousefpour et al. (2021).
LiDAR	Light Detection and Ranging. A sensor used in AV systems that produces 3D point cloud measurements.

NPC	Non-Player Character. AI-controlled vehicles and pedestrians spawned in CARLA simulation scenes.
AV	Autonomous Vehicle.
SDG	Sustainable Development Goal, as defined by the United Nations.

1.4 Overview

Chapter 2 provides an overall description of the system including a comparison with existing systems, major functions, user characteristics, constraints, assumptions, and implementation schedule. Chapter 3 provides the specific design details structured for an embedded or system-oriented project: external interface requirements, high-level architectural design, hardware and software requirements with justification, risk mitigation plan with alternative solutions, cost-benefit analysis, performance requirements, design constraints, software system attributes, and other requirements. Chapter 4 contains supporting information and appendices.

The system is implemented in Python 3.11. The privacy enforcement engine uses PyTorch 2.5.1 and Opacus 1.4.1. The simulation environment is CARLA 0.9.x. All four modules are integrated through a shared GitHub repository with defined file-based interfaces.

2. OVERALL DESCRIPTION

2.1 Product Perspective

The Privacy-Enhanced Synthetic AV Data Generation Pipeline is a research system that integrates two previously separate technology domains: AV simulation platforms and differential privacy enforcement. The table below compares this system against the closest existing alternatives to establish the product context.

Table 2-1: Product Comparison

System	Privacy Guarantee	Multi-model AV Data	Calibration Protected	Attack Validated
CARLA (Dosovitskiy et al., 2017)	None	Yes	No	No
AirSim (Shah et al., 2018)	None	Yes	No	No
NVIDIA DRIVE Sim	None	Yes	No	No
DP-GAN (Xie et al., 2018)	Formal epsilon-DP	No (MNIST only)	No	No
DP-CGAN (Torkzadehmahani et al., 2019)	Formal epsilon-DP	No (MNIST only)	No	No
This System	Formal epsilon-DP at calibration and training	Yes (Camera, LiDAR, Radar)	Yes (Laplace mechanism)	Yes (Shadow-model MIA)

Existing AV simulation platforms treat data generation as a realism and scalability problem. None address what information is retained from the real-world data used to calibrate them. Existing differentially private GAN implementations address gradient-level memorization in neural network training but have only been validated on single-modal image benchmarks such as MNIST and CIFAR-10. No published work applies epsilon-DP to the calibration and training stages jointly for multi-modal AV sensor data, and no published work validates the result against domain-specific attack tests. This system fills both gaps.

2.1.1 System Interfaces

The system runs as a command-line pipeline on Windows 11 during development and on Ubuntu 22.04 for Phase 3 cloud generation. The four modules communicate through file-based interfaces stored in the shared outputs/ directory. The primary system-level interfaces are listed below.

- **calibration_stats.json:** output from Privacy Integration module, input to CARLA simulation module.

- **camera_frames.pt, lidar_frames.pt, labels.pt:** output from CARLA simulation module, input to Utility Evaluation module
- **audit_report.json, budget_log.csv:** output from Privacy Integration module, input to Risk and Compliance module.
- mia_results.json: output from Risk and Compliance module, shared with all members.
- **compliance_report.json:** output from Risk and Compliance module, delivered to supervisors and end users.

2.1.2 User Interfaces

The system is operated through Python command-line scripts. All configurable parameters are set in config.yaml before execution. No graphical user interface is provided for the research prototype. The planned CLI pipeline tool will expose all four stages through a single-entry point command:

python pipeline.py --input company_data.json --epsilon 3.0 --output results/

2.1.3 Hardware Interfaces

The system interfaces with NVIDIA GPU hardware through PyTorch's CUDA backend. Development runs use an NVIDIA GeForce RTX 3050 Ti Laptop GPU with 4GB VRAM and CUDA 12.5 driver support. Phase 3 mass generation uses AWS EC2 GPU instances (G4 or G5 family). CARLA scene rendering during Phase 3 requires a dedicated NVIDIA GPU on the Ubuntu server. No embedded sensors or physical AV hardware components are part of this system.

2.1.4 Software Interfaces

- **PyTorch 2.5.1 (CUDA 12.1 build):** deep learning framework for all model training and evaluation
- **Opacus 1.4.1:** epsilon-DP enforcement library with built-in RDP accountant
- **CARLA 0.9.x:** AV simulation environment operated by IT22541284.
- **Waymo Open Dataset SDK:** used to extract aggregate statistical properties during Phase 3 calibration.
- **NumPy, Pandas, scikit-learn, scipy, matplotlib:** data processing, metrics computation, and result visualization.
- **PyYAML:** configuration file parsing
- **GitHub:** version control and inter-member file handoff
- **AWS S3:** large dataset storage for Phase 3 cloud generation batches

2.1.5 Communication Interfaces

All inter-module communication is file-based through the shared GitHub repository. For Phase 3, large tensor dataset batches are transferred between local machines and AWS instances using SSH and S3 protocols. No real-time messaging, REST APIs, or network sockets are required between system modules.

2.1.6 Memory Constraints

Opacus requires per-sample gradient storage before clipping, which approximately doubles GPU memory usage compared to standard batch gradient computation. On the RTX 3050 Ti with 4GB VRAM, the training batch size is limited to thirty-two during development runs. Phase 3 cloud instances target batch sizes of 64 to 128. Synthetic dataset storage across all three ablation configurations is estimated at 1TB on AWS S3. Budget logs and audit reports are small JSON and CSV files that impose no meaningful storage constraint.

2.1.7 Operations

Normal operations include running the Privacy-Aware Calibration Module to produce privatized calibration_stats.json, configuring and running CARLA batch generation using the privatized statistics, running the DP Training Wrapper across three ablation configurations, training the AV perception model on synthetic datasets, running the shadow-model MIA calibration study, and generating the GDPR compliance report. Special operations include automatic training halt when the RDP accountant reaches the epsilon ceiling of 10.0, and the ModuleValidator auto-fix procedure that replaces BatchNorm layers with GroupNorm before DP training begins.

2.1.8 Site Adaptation Requirements

Development environment: Windows 11, Miniconda, Python 3.11.15, PyTorch 2.5.1 cu121 build, Opacus 1.4.1, NVIDIA CUDA 12.5 driver.

Phase 3 cloud environment: Ubuntu 22.04 LTS, AWS EC2 GPU instance (G4 or G5 family), CARLA 0.9.x in headless mode.

The system does not use platform-specific APIs beyond PyTorch's CUDA backend, so deployment on Ubuntu requires only CUDA driver verification and environment setup using the provided requirements.txt and config.yaml.

2.2 Product Functions

The major functions of the system are grouped by components.

Privacy Integration Component (IT22309556):

- Extract aggregate statistics from real-world AV data in memory without writing raw data to disk.
- Apply the Laplace mechanism to each aggregate statistic before it is passed to the simulator.
- Enforce epsilon-DP during generative model training using Opacus DP-SGD with per-sample gradient clipping and Gaussian noise injection.
- Track the cumulative privacy budget per training step using the RDP accountant.
- Halt training automatically when the configured epsilon ceiling is reached.
- Produce structured audit reports and per-step CSV budget logs.
- Execute the shadow-model membership inference attack with a two-feature signal comprising confidence score and per-sample loss.

- Produce the empirical epsilon-to-attack-resistance calibration curve across three configurations and a no-DP baseline.

Dataset Generation Component (IT22541284):

- Configure CARLA simulation scenes using privatized calibration statistics.
- Generate labelled multi-sensor synthetic data batches including camera RGB frames, LiDAR point clouds, and radar returns with automatic ground-truth bounding boxes and segmentation labels.
- Parameterize scenes across weather conditions, lighting, time of day, and traffic density as specified in the calibration statistics.
- Package output data in the agreed PyTorch tensor format with accompanying metadata files.

Utility Evaluation Component (IT22110220):

- Establish a baseline mAP score by running a pre-trained AV perception model on a real-world dataset such as Waymo.
- Train AV perception models on the synthetic datasets generated under each of the three epsilon configurations; measure mAP and FID scores for each configuration and compare them against the real-data baseline.
- Produce the privacy-utility trade-off analysis showing how mAP and FID vary across epsilon values.

Risk and Compliance Component (IT22066916):

- Develop and execute shadow-model MIA and re-identification attack scripts against the synthetic datasets generated under each epsilon configuration.
- Compile attack success rates, AUC values, and interpretation per configuration.
- Map each system controls specific GDPR articles and NIST.
- Produce the integrated compliance report combining attack results, budget metrics, and regulatory mapping.

2.3 User Characteristics

The system is currently used by the four project members during development and testing. All members are familiar with Python, PyTorch, differential privacy, and autonomous vehicle research. They run the system through command-line scripts and review outputs in JSON and CSV formats.

Academic supervisors review the generated audit reports, compliance records, budget logs, and privacy certificates. Their role is limited to examining the output and documentation produced by the system. They do not directly operate the pipeline.

In a production environment, the system is intended for machine learning engineers and data infrastructure teams working in autonomous vehicle companies and research institutions. These users use the CLI pipeline to generate GDPR-compliant synthetic training data on a scale. The privacy certificate is one of the main outputs provided by the system.

2.4 Constraints

- The total cumulative epsilon across calibration and training must not exceed epsilon = 10.0 in any experimental run. This is enforced automatically by the RDP accountant with a hard training halt.
- The calibration step is allocated a maximum of epsilon = 2.0, which represents 20 percent of the total budget. This constraint is enforced programmatically in the LaplaceCalibrationModule class with a ValueError raised on violation.
- Only aggregate statistics derived from the Waymo Open Dataset may be used for calibration. Raw frames and point clouds must be processed in memory only and must never be written to disk. Dataset access is governed by the Waymo Dataset License Agreement for non-commercial academic research.
- The DP enforcement must use Opacus 1.4.1 with PyTorch 2.5.1. Python 3.14 is not supported because PyTorch does not publish wheels for it.
- Development training runs are constrained to the NVIDIA RTX 3050 Ti with 4GB VRAM. Phase 3 cloud runs must stay within the LKR 32,000 compute budget allocated for approximately 200 GPU hours on AWS.
- The system must not store personal data beyond the duration of computation. All privacy controls must satisfy GDPR Article 25 (data protection by design) and Sri Lanka PDPA No. 9 of 2022.
- Opacus does not support nn.BatchNorm layers. All model architectures must pass ModuleValidator.is_valid() and be repaired with ModuleValidator.fix() before DP training begins.

2.5 Assumptions and Dependencies

- CARLA 0.9.x is available and operational on the Ubuntu server used for Phase 3 batch generation.
- Waymo Open Dataset access is available under the research license for the full duration of the project.
- IT22541284 delivers CARLA synthetic dataset batches in the agreed PyTorch tensor format before Phase 3 AV model training begins.
- IT22110220 establishes a real-data mAP baseline using the Waymo dataset before synthetic model training begins.
- IT22066916 receives the audit reports from IT22309556 before running Phase 3 MIA tests.
- AWS cloud GPU instances are accessible within the allocated compute budget throughout Phase 3.
- The generative model architecture selected by the group is compatible with Opacus after ModuleValidator.fix() is applied.
- The shared GitHub repository is accessible to all four group members throughout the project duration.

2.6 Apportioning of Requirements

The system is built and validated across four phases according to the project schedule. Phase 1 covers Weeks 1 to 3 and includes literature review, library selection, environment setup, architecture design, and interface specification. Phase 2 covers Weeks 4 to 7 and includes implementation of all four Privacy Integration sub-modules, CARLA scene configuration, and baseline utility model setup. Phase 3 covers Weeks 8 to 10 and includes mass synthetic data generation across all three epsilon configurations, AV perception model training, and MIA calibration study execution. Phase 4 covers Weeks 11 to 14 and includes privacy-utility trade-off analysis, compliance reporting, and the final report.

3. SPECIFIC REQUIREMENTS

3.1 External Interface Requirements

3.1.1 User Interfaces

The system exposes a command-line interface. All parameters are controlled through config.yaml before execution. The table below lists the primary entry points.

Table 3-1: Primary Entry Points

Script	Module	Description
run_calibration.py	Privacy Integration	Runs the Laplace calibration module. Outputs calibration_stats.json and calibration audit_log.json to outputs/calibration/.
run_dp_training.py	Privacy Integration	Runs the DP Training Wrapper across all three ablation configurations. Outputs budget logs and audit reports per configuration to outputs/budget_logs/ and outputs/audit_reports/.
run_calibration_study.py	Privacy Integration	Runs the shadow-model MIA against all epsilon configurations and the no-DP baseline. Outputs calibration_curve_data.json to outputs/results/.
scene_config.py	Dataset Generation	Loads calibration_stats.json and generates the CARLA scene configuration parameters for simulation.
train_av_model.py	Utility Evolution	Trains the AV perception model on a specified synthetic dataset batch. Outputs mAP and FID scores.
gdpr_checklist.py	Risk and Complience	Generates the GDPR compliance report by combining audit report and MIA results data.

3.1.2 Hardware Interfaces

The system interfaces with NVIDIA GPU hardware through PyTorch's CUDA backend. Device selection is automatic at runtime: CUDA is used if a compatible device is detected, otherwise CPU is used with a console warning. Opacus per-sample gradient computation runs on GPU. CARLA batch rendering during Phase 3 requires a dedicated NVIDIA GPU with at least 8GB VRAM on the Ubuntu server for efficient scene generation.

3.1.3 Software Interfaces

Opacus wraps the PyTorch optimizer through the PrivacyEngine.make_private_with_epsilon() API, which auto-computes the noise multiplier from the target epsilon, epochs, and batch size. CARLA is interfaced through its Python client library using TCP connection to the CARLA server process. The Waymo Open Dataset is accessed through the official TensorFlow-based SDK for in-memory aggregate extraction only. All inter-module data interfaces use JSON and PyTorch .pt tensor file formats as specified in Section 3.2.1.

3.1.4 Communication Interfaces

Modules communicate through file reads and writes on the shared GitHub repository. For Phase 3, synthetic dataset batches (estimated 500GB per epsilon configuration) are transferred between local machines and AWS EC2 instances using standard AWS S3 upload and download commands. No real-time or streaming communication interfaces are required between modules.

3.2 Architectural Design

3.2.1 High Level Architectural Design

The system follows a linear four-stage pipeline architecture. Privacy enforcement is applied upstream of data generation rather than as a post-processing step on completed outputs. The table below shows the pipeline stages, ownership, inputs, and outputs.

Table 3-2: The Pipeline Stages

Stage	Component	Owner	Input	Output
Privacy-Aware Calibration	Laplace Calibration Module	IT22309556	Waymo aggregate statistics processed in memory	calibration_stats.json, calibration_audit_log.json
Simulation and Dataset Generation	CARLA Scene Runner	IT22541284	calibration_stats.json	camera_frames.pt, lidar_frames.pt, labels.pt, metadata.json
DP Generative Model Training	DP Training Wrapper	IT22309556	Synthetic dataset batches from stage 2	Trained model weights, budget_log.csv, audit_certificate.json, audit_report.json
AV Model Training and Utility Evaluation	AVPerception Model, MetricsEvaluator	IT22110220	Synthetic dataset batches from Stage 2	MAP scores, FID scores, trade-off analysis

Privacy Validation	ShadowMIAAttack	IT22066916 and IT22309556	Trained model, synthetic dataset, audit report	mia_results.json,calibration_curve_data.json
Compliance Reporting	GDPRComplianceChecker	IT22066916	Audit report, MIA results	compliance_report.json, privacy_certificate.json

The Privacy Integration component is active at two points in the pipeline. It protects the calibration channel (Stage 1) by applying the Laplace mechanism to aggregate statistics before they configure the simulator. It also protects the training channel (Stage 3a) by enforcing DP-SGD through Opacus during generative model training. This dual placement is the primary architectural design decision that differentiates this system from existing AV data generation platforms.

The four Privacy Integration sub-modules and their internal data flow are as follows.

- **Privacy-Aware Calibration Module:** receives aggregate statistics from the Waymo SDK in memory, applies Laplace noise scaled to sensitivity divided by epsilon_calibration, outputs privatised statistics to calibration_stats.json, and discards all raw data without writing it to disk.
- **DP Training Wrapper:** wraps the PyTorch optimizer using Opacus PrivacyEngine, clips each per-sample gradient to L2 norm C using functorch, adds Gaussian noise scaled to C multiplied by sigma to the averaged clipped gradients, and passes the noised gradient to the optimizer at each training step.
- **Privacy Budget Accountant:** embedded in the DP Training Wrapper, uses the RDP composition framework built into Opacus to accumulate the epsilon cost of each training step, and raises BudgetExceededError to halt training when the cumulative epsilon reaches the configured ceiling.
- **Privacy Audit Interface:** packages the final epsilon value, clipping norm C, noise multiplier sigma, total training steps, and the full budget log into a structured audit_report.json file that is passed to IT22066916 before MIA testing begins.

The ShadowMIAAttack component implements the four-stage shadow model protocol from Shokri et al. (2017) with a two-feature extension. The standard single-confidence-score attack signal is extended to include per-sample loss, because per-sample loss captures gradient-level memorization more directly than output confidence for generative model architectures. The attack classifier input dimension is 2 (confidence, loss) rather than 1.

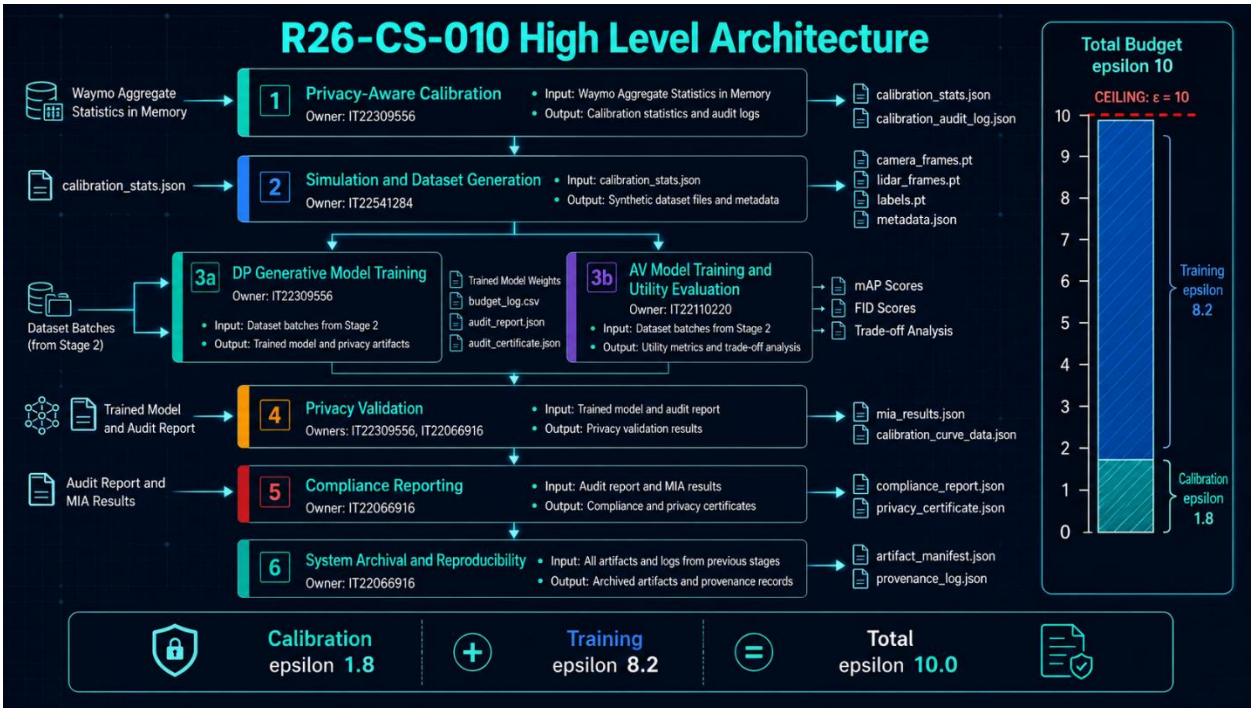


Figure 3-1: High Level Architecture

3.2.2 Hardware and Software Requirements with Justification

Table 3-3: Hardware and Software Requirements

Requirements	Component	Justification
Python 3.11.15 via Miniconda	All modules	PyTorch 2.5.1 wheel distributions support Python 3.9 to 3.12 only. The system default Python 3.14 on development machines is incompatible. Miniconda enables a separate 3.11 environment without system conflicts.
Pytorch 2.5.1 (CUDA 12.1 build)	Privacy Integration, Utility Evaluation	Standard deep learning framework for AV research. Native Opacus integration through functorch for per-sample gradient computation. CUDA 12.1 build is forward-compatible with CUDA 12.5 drivers.
Opacus 1.4.1	Privacy integration	Provides native per-sample gradient computation without model architecture changes, RDP accountant for tight epsilon bounds, and automatic training halt at the configured epsilon ceiling. Apache 2.0 licence with no cost.

NVIDIA GPU with CUDA 12.x	Privacy Integration, Utility Evaluation	Per-sample gradient computation in Opacus is parallelisable across GPU cores. Training time on CPU would be prohibitive for the three ablation runs. Minimum 4GB VRAM for development, 8GB+ recommended for Phase 3.
CARLA 0.9.x	Dataset Generation	Open-source AV simulator that produces camera, LiDAR, and radar sensor outputs with automatic ground-truth labelling. Supports parameterized weather, lighting, and traffic configuration. MIT license.
Waymo Open dataset SDK	Privacy Integration (Phase 3)	Provides access to 1,950 driving segments across diverse weather and geographic conditions for aggregate calibration statistics. Accessed under the Waymo research licence. Only aggregate-level queries are made.
AWS EC2 G4 or G5 GPU instance	Dataset Generation, Privacy Integration (Phase 3)	Local RTX 3050 Ti (4GB VRAM) is not sufficient for large-batch CARLA rendering or extended DP training runs. AWS provides scalable GPU computes within the LKR 32,000 budget allocation.
GitHub repository	All modules	Shared repository enables inter-member file handoff, version control, and audit trail of all code changes. Free for academic use.

3.2.3 Risk Mitigation Plan with Alternative Solution Identification

Table 3-4: Risk Mitigation Plan

Risk	Likelihood	Impact	Mitigation	Alternative
Privacy budget exhausted before training completes	Medium	High	Pre-calculated expected epsilon consumption per phase using RDP composition formulas before the full run. Use subsampling amplification to reduce the per-step epsilon cost.	Reduce the number of training epochs or increase batch size to lower per-step epsilon cost. Switch to a higher target epsilon for that configuration and document the change.
DP noise reduces synthetic data quality below the 15% mAP degradation threshold	Medium	High	Run all three ablation configurations (epsilon = 1.0, 3.0, 8.0) before committing to mass generation. Select the configuration with the best	Increase the target epsilon for Config A if mAP degradation exceeds 15%. Document the trade-off result in the thesis rather than forcing a failing configuration.

			privacy-utility balance based on ablation results.	
Opacus incompatibility with the generative model architecture	Low	Medium	Run ModuleValidator.is_valid() on every model before DP training. Use ModuleValidator.fix() to auto-replace unsupported layers such as BatchNorm with GroupNorm.	Fall back to TensorFlow Privacy if the group adopts a TensorFlow-based generative pipeline. TF Privacy provides the same gradient clipping mechanism with a different API.
CARLA unavailable or unstable during Phase 3 batch generation	Low	High	Assess the full CARLA pipeline on small batches before Phase 3 mass generation. Save checkpoints between scenario batches so a crash does not restart the full generation.	Use AirSim as an alternative simulator. The calibration_stats.json interface is simulator-agnostic and can configure AirSim using the same field mapping.
MIA attack results are ambiguous or show very high variance across shadow models	Low	Medium	Use both shadow-model and likelihood-ratio MIA variants. Compare empirical results against the formal epsilon bound from the RDP accountant to establish whether the bounds are tight.	Increase n_shadow_models from 4 to 8 for more statistically robust attack estimates. Report confidence intervals alongside mean success rates.
AWS compute budget exceeded during Phase 3	Medium	Medium	Evaluate batch generation throughput on small samples before full runs. Monitor GPU utilisation and adjust batch size to maximise frames-per-hour efficiency.	Reduce dataset size per configuration or run the three configurations sequentially rather than in parallel to stay within the LKR 32,000 budget.
Waymo Dataset licence restricts aggregate query use	Low	High	Calibration uses only aggregate statistical queries. No raw frames are stored. The calibration process is fully documented and auditable against the licence terms.	Use KITTI or nuScenes as alternative calibration sources. Both are publicly available for academic research use and cover comparable driving conditions.
A group member's deliverable is delayed and blocks a downstream module	Medium	Medium	Define handoff dates in the project Teams Planner. Use the generate_sample_dataset.py placeholder tensors to unblock downstream members during development.	Proceed with placeholder data for development and integration testing. Replace with real outputs when available. The shared dataset_loader.py manages both placeholder and real formats identically.

3.2.4 Cost Benefit Analysis for the Proposed Solution

The cost-benefit analysis is presented in two parts: the project research costs, and the commercial benefits of the system design.

Table 3-5: Project Research Costs

Cost Item	Estimated	Justification
Cloud GPU compute (AWS, approximately 200 hours)	**LKR 32,000**	DP training runs slower than standard training due to per-sample gradient computation. Multiple ablation experiments across three epsilon configurations are required before Phase 3 mass generation.
Cloud storage (1TB, AWS S3)	**LKR 8,000**	Storage for synthetic dataset batches, model checkpoints, and privacy budget logs across all experimental runs and three epsilon configurations.
Research paper access (IEEE Xplore, ACM Digital Library)	**LKR 8,000**	Key papers on differentially private GANs and AV data privacy are behind paywalls. Required for literature review and methodology validation.
Opacus and PyTorch (open-source, Apache 2.0)	**LKR 0**	No licence fees. Both libraries are freely available.
CARLA simulator (open-source, MIT licence)	**LKR 0**	No licence fees.
Development hardware (existing group laptops)	**LKR 0**	No new hardware purchases required. All developments run on existing RTX 3050 Ti machines.
Human effort (four members, 14 weeks)	**LKR 0**	Covered under the academic project scope.
Total	**LKR 48,000**	

Commercial Benefits:

- **Mitigation of Regulatory Risk:**

Autonomous vehicle developers who employ this system are able to demonstrate GDPR compliance by means of a verifiable privacy certificate, a capability of considerable significance given the current regulatory climate. The European Data Protection Board imposed in excess of €1.6 billion in GDPR-related financial penalties across member states during 2022 alone. Possession of an auditable privacy certificate substantially diminishes an organization's legal exposure.

- **Reduction of Data Licensing Expenditure:**

Organizations that produce synthetic datasets through this system are not required to allocate substantial financial resources toward licensing real-world driving footage. Such licensing agreements frequently amount to hundreds of thousands of dollars annually. The provision of certified synthetic data at a reduced cost constitutes the primary value proposition of this system.

- **Regulatory Defensibility:**

The system autonomously generates a comprehensive, documented audit trail comprising budget_log.csv, audit_report.json, and compliance_report.json which regulatory authorities may examine at any point in time. No existing autonomous vehicle synthetic data generation platform presently offers a comparable capability.

- **Contribution to Academic Research:**

The empirical epsilon-to-attack-resistance calibration curve produced by the ShadowMIAAttack component furnishes researchers with a measurable and empirically testable means of determining whether formal epsilon bounds reliably predict real-world attack resistance within the autonomous vehicle domain a question that has not been addressed in any previously published work pertaining to multi-modal autonomous vehicle data.

- **Commercialization Pathway:**

The system is inherently well-suited to a Software-as-a-Service model, wherein organizations submit their calibration configuration and receive in return a certified synthetic dataset accompanied by a comprehensive audit report. A distinct business-to-business licensing avenue also exists, whereby the epsilon-DP pipeline may be integrated directly into established simulation platforms such as CARLA or NVIDIA DRIVE Sim under the terms of an annual licensing agreement.

3.3 Performance Requirements

- **Privacy-Aware Calibration Module Responsiveness Requirements:**

The Privacy-Aware Calibration Module is likewise subject to stringent performance expectations. All aggregate statistical queries must conclude within five seconds on development hardware. The full shadow-model membership inference attack calibration study encompassing four shadow models evaluated across three epsilon configurations in addition to a non-private baseline must reach completion within thirty minutes on the same hardware environment.

- **RDP Accountant Tracking Overhead:**

The computational overhead attributable to Rényi Differential Privacy accountant tracking is, by its nature, confined to scalar arithmetic performed at each training step. Consequently, this process is expected to contribute a negligible increment to per-step runtime.

- **CARLA Batch Generation Efficiency:**

CARLA batch generation must proceed at a sufficient throughput to produce the complete synthetic dataset within the allocated budget of two hundred GPU-hours on AWS infrastructure.

- **Perception Model Performance Threshold:**

Any autonomous vehicle perception model trained exclusively on synthetic data must attain mean Average Precision scores within fifteen percent of the real-data baseline established on the Waymo dataset.

3.4 Design Constraints

- **Implementation Language:**

Python 3.11 constitutes the mandated implementation language for this project. Python 3.14 is explicitly excluded from consideration, as PyTorch wheel distributions do not yet extend support to that version.

- **Differential Privacy Enforcement:**

Epsilon-DP enforcement is to be achieved exclusively through Opacus 1.4.1, utilizing the Rényi Differential Privacy accountant. TensorFlow Privacy and its associated moments accountant are not permitted alternatives, as they yield comparatively less precise epsilon bounds.

- **Per-Sample Gradient Clipping:**

Per-sample gradient clipping must be implemented solely through Opacus's native functorch integration. The manual implementation of per-sample gradient loop routines is expressly prohibited.

- **Calibration Data Handling:**

All calibration statistics must be computed at the aggregate level without exception. Individual frame-level Waymo data must be discarded from memory prior to any transmission to the LaplaceCalibrationModule. This requirement is structurally enforced through the module's class design, which accepts exclusively a dictionary of numeric aggregates as its input parameter.

- **Reproducibility:**

Every random seed, noise configuration, and epsilon value must be recorded automatically in conjunction with their corresponding results, such that any given run may be reproduced precisely from its config.yaml file and associated seed log.

- **Cross-Platform Compatibility:**

The system must operate identically across both Windows 11, which serves as the designated development environment, and Ubuntu 22.04, which is employed for Phase 3 cloud-based execution without necessitating any platform-specific modifications to the codebase.

- **Model Architecture Validation:**

Given that Opacus does not provide support for nn.BatchNorm layers, all model architectures must undergo validation by means of ModuleValidator.is_valid() and, where necessary, remediation through ModuleValidator.fix() prior to the commencement of differential privacy training.

- **Personal Data Prohibition:**

It is an absolute and non-negotiable requirement that no personal data be written to disk at any stage of the pipeline. This constraint is embedded directly within the structural design of the LaplaceCalibrationModule to ensure its unconditional enforcement.

3.5 Software System Attributes

3.5.1 Reliability

- **Epsilon Budget Enforcement:**

The Rényi Differential Privacy accountant bears an unconditional obligation to terminate training prior to any breach of the configured epsilon ceiling. This requirement admits no exceptions, including circumstances in which a run is interrupted prematurely as a consequence of hardware failure or process termination.

- **Atomic Budget Logging:**

Budget logs are committed atomically upon the conclusion of each individual training step. This mechanism ensures that a run terminated unexpectedly under any condition cannot produce a partially written or otherwise corrupt log file.

- **Unit Testing Requirements:**

Each of the four Privacy Integration sub-modules must be accompanied by its own independent suite of unit tests, residing within the designated privacy_integration/tests/ directory. The successful completion of every such test constitutes a mandatory prerequisite for the initiation of any Phase 3 run; no exceptions to this condition are permissible.

3.5.2 Availability

- **Availability and Output Integrity:**

Given that this system constitutes a research pipeline rather than a live production service, availability requirements pertain principally to the integrity of output files rather than to operational uptime. Every output file must be written in its entirety prior to process termination; partially saved files are not permissible under any circumstances.

- **Interrupted Run Handling:**

In the event that a run is interrupted prior to completion, any resultant incomplete output files must be unambiguously marked with a timestamp reflecting the point of interruption. Such files must under no circumstances be silently ingested by downstream modules as though they represent complete and valid outputs.

- **Repository Accessibility:**

The shared GitHub repository must remain accessible to all four members of the project team throughout the full duration of the project.

3.5.3 Security

Raw calibration data must never touch disk inside the Privacy Integration component. Any intermediate computation over raw data happens in memory only and gets discarded immediately after use. Model weights, budget logs, and audit reports must be stored with file-system access controls that lock down read and write permissions to the research group. The shadow-model MIA attack scripts must not leave the team until the project is fully completed and submitted.

On the privacy side, because epsilon-DP is enforced throughout training, any adversary who somehow gets hold of the model weights or the synthetic dataset is mathematically limited in what they can infer about any individual bounded directly by the configured epsilon value.

3.5.4 Maintainability

- **Modular Architecture Requirements:**

Each of the four Privacy Integration sub-modules must be constructed as a discrete, independently importable Python module, accompanied by its own dedicated unit test suite. The replacement or substitution of any individual sub-module must not necessitate modifications to the remaining three; only the relevant interface call requires updating in such an eventuality.

- **Configuration Management:**

All configurable parameters must reside exclusively within config.yaml and shared/constants.py. The direct embedding of numeric values including those pertaining to epsilon, clipping norm, or operational thresholds within module code is strictly prohibited.

- **Anticipated Module Evolution:**

Certain classes are expected to undergo revision as the project advances through subsequent phases. Specifically:

- The **LaplaceCalibrationModule** may require adjustment to its sensitivity parameters upon the introduction of real Waymo data during Phase 3.
- The **DPTrainingWrapper** is anticipated to necessitate model-specific remediation to accommodate new generative architectures originating from IT22541284.
- The **ShadowMIAAttack** attack classifier architecture may be subject to revision based upon the characteristics of real generative model outputs made available during Phase 3.

3.6 Other Requirements

3.6.1 Privacy Requirements

- The total cumulative epsilon for calibration and training must stay at or below 10.0 for all runs, verified using the RDP accountant log after each run.
- The epsilon during calibration must stay below 2.0 for all runs.
- Controlled through LaplaceCalibrationModule where a ValueError exception is raised if the configured epsilon equals or exceeds the threshold.
- The shadow model's MAI accuracy score for the generated data must be less than 55 percent. The goal is around 50 percent, which implies random guessing. Empirically verified using IT22066916 in phase 3.
- The re identification attack accuracy score on the generated driving trajections must be less than 10 percent. Empirically verified using IT22066916 in Phase 3.

3.6.2 Regulatory Requirements

- **GDPR Article 5(1)(b) - Purpose limitation:**

Waymo data is used only for aggregate calibration statistics. It is not reused for generative model training.

- **GDPR Article 5(1)(c) - Data minimization:**

Only seven aggregate statistical queries are made against Waymo data. No individual records are retained.

- **GDPR Article 17 - Right to erasure:**

No individual records are stored anywhere in the pipeline. Epsilon-DP prevents recovery of individual data from model weights.

- **GDPR Article 25 - Privacy by design:**

Epsilon-DP is enforced during calibration and training, not applied as a post-processing step to already-generated data.

3.6.3 Reproducibility Requirements

- All random seeds shall be logged in the budget and audit logs for all experiments.
- The experiment config.yaml shall be stored along with the results files.
- A single experiment can be reproduced solely based on the stored config.yaml and seed log information.

4. REFERENCES

[1] C. Dwork, “Differential Privacy,” in Proc. 33rd Int. Colloq. Automata, Languages and Programming (ICALP), Berlin, Germany: Springer, 2006, pp. 1-12.

[2] M. Abadi et al., “Deep Learning with Differential Privacy,” in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2016, pp. 308-318.

[3] A. Yousefpour et al., “Opacus: User-Friendly Differential Privacy Library in PyTorch,” arXiv preprint arXiv:2109.12298, 2021.

[4] A. Dosovitskiy et al., “CARLA: An Open Urban Driving Simulator,” in Proc. 1st Annu. Conf. Robot Learning (CoRL), 2017, pp. 1-16.

[5] R. Shokri et al., “Membership Inference Attacks against Machine Learning Models,” in Proc. IEEE Symp. Security and Privacy (S&P), 2017, pp. 3-18.

[6] L. Xie et al., “Differentially Private Generative Adversarial Network,” arXiv preprint arXiv:1802.06739, 2018.

[7] R. Torkzadehmahani et al., “DP-CGAN: Differentially Private Synthetic Data and Label Generation,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW), 2019.

[8] I. Goodfellow et al., “Generative Adversarial Networks,” in Advances in Neural Information Processing Systems (NeurIPS), 2014, pp. 2672-2680.

[9] N. Carlini et al., “Extracting Training Data from Large Language Models,” in Proc. USENIX Security Symp., 2021, pp. 2633-2650.

[10] A. Narayanan and V. Shmatikov, “Robust De-anonymization of Large Sparse Datasets,” in Proc. IEEE Symp. Security and Privacy (S&P), 2008, pp. 111-125.

[11] European Union, “General Data Protection Regulation (GDPR),” Regulation (EU) 2016/679, Official Journal of the European Union, 2016.

5. APPENDICES

Appendix A - Module Interface Specification

File	Format	Produced by	Consumed by	Description
calibration_stats.json	JSON	IT22309556	IT22541284	Privatized aggregate scene statistics. Seven Laplace-noised numeric fields. Safe to share with the simulator operator.
calibration_audit_log.json	JSON	IT22309556	IT22066916	Audit trail of queries made against Waymo data, including the sensitivity and noise applied to each query.
budget_log.csv	CSV	IT22309556	IT22309556, IT22066916	Per-step epsilon log. Columns: step, cumulative_eps, loss. One row per training step across all epochs.
audit report.json	JSON	IT22309556	IT22066916	Final epsilon, clipping norm C, noise multiplier sigma, total training steps, halted_early flag, and budget_respected flag per experimental run.
camera frames.pt	PyTorch tensor	IT22541284	IT22110220	Camera RGB frames. Shape: (N, 3, 256, 256).
lidar_frames.pt	PyTorch tensor	IT22541284	IT22110220	LiDAR point clouds. Shape: (N, 1024, 4) representing x, y, z, and intensity per point.
labels.pt	PyTorch tensor	IT22541284	IT22110220	Ground-truth object labels. Shape: (N, 10, 5) representing class, x, y, w, h for up to 10 objects per frame.
metadata.json	JSON	IT22541284	IT22110220	Dataset metadata includes config tag, epsilon value, sensor shapes, and audit report reference.
mia_results.json	JSON	IT22066916	All members	MIA attack success rate, AUC, and interpretation text per epsilon configuration.
calibration_curve_data.json	JSON	IT22309556	All members	Empirical epsilon-to-attack-resistance calibration curve. Contains data points for all

				configurations and the no-DP baseline.
compliance_report.json	JSON	IT22066916	All members, supervisors	Integrated compliance report mapping each system control to specific GDPR articles and PDPA provisions.
privacy_certificate.json	JSON	IT22309556	End users, regulators	Full privacy certificate including configuration, MIA results, compliance notes, and overall PASSED or FAILED status.

Appendix A: Module Interface Specification

Appendix B - Privacy Budget Allocation

Pipeline Stage	Epsilon Allocation	Percentage of Total	Notes
Calibration step using Laplace mechanism	epsilon less than 2.0	20%	Controlled by LaplaceCalibrationModule. Logged in calibration_audit_log.json under calibration_epsilon_spent.
DP Training - Config A	epsilon = 1.0	10%	Strong privacy. Noise multiplier approximately 1.97. High noise level. Expected utility degradation above other configs.
DP Training - Config B	epsilon = 3.0	30%	Moderate privacy. Noise multiplier approximately 1.01. Primary ablation configuration for Phase 3.
DP Training - Config C	epsilon = 8.0	80%	Weak privacy. Noise multiplier approximately 0.67. Low noise level. Expected utility close to non-DP baseline.
Total pipeline ceiling	epsilon less than or equal to 10.0	100%	Hard ceiling enforced by the RDP accountant. Training halts automatically when this value is reached.

Appendix B: Privacy Budget Allocation

Appendix C - SDG Alignment

- **SDG 9 (Industry, Innovation, and Infrastructure):** The system provides privacy-preserving AI data infrastructure for the AV industry. It enables model training at scale without requiring real-world data licensing or GDPR compliance overhead.
- **SDG 11 (Sustainable Cities and Communities):** Certified privacy-compliant AV training data supports the development of safer and more trustworthy autonomous transport systems for urban environments.
- **SDG 16 (Peace, Justice, and Strong Institutions):** The auditable pipeline with its budget logs, audit reports, and compliance mapping demonstrates that AI data governance can be made accountable, verifiable, and institutionally transparent.

Appendix D - Repository Structure

- Privacy-Integration/ owned by IT22309556: subdirectories calibration/, training/, attack/, and tests/
- Dataset-Generation/ owned by IT22541284: subdirectories carla_setup/, simulation/, sensors/, and tests/
- Utility-Evaluation/ owned by IT22110220: subdirectories models/, evaluation/, training/, and tests/
- Risk-Compliance/ owned by IT22066916: subdirectories attacks/, compliance/, and tests/
- shared/ used by all members: constants.py, utils.py, and dataset_loader.py
- docs/ for all project documents: subdirectories proposal/, progress_reports/, handoff_docs/, and final_report/
- config.yaml at the repository root: global configuration file for epsilon values, dataset parameters, and output paths
- requirements.txt at the repository root: Python dependency specification including all library versions.